

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР1: Реализация REST API на основе boilerplate

Срок: 11.03.2026

Задание:

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и [мой](#))
2. С учётом имеющихся данных по результатам Д31 и Д32 реализуйте:
 - модели
 - представления
 - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

Я работал над вариантом - Приложение для бронирования столиков в ресторанах. В прошлом домашнем задании была уже составлена схема API, поэтому написание сущностей было намного проще – чтобы создать БД благодаря TypeORM нам не нужно писать SQL-запросы вручную, TypeScript сделает это за нас. Основные сущности:

Пользователи (User): хранение данных клиентов и их ролей.

Регионы и Города (Region, City): географическая привязка ресторанов.

Рестораны (Restaurant): информация о заведениях, категориях цен и кухнях.

Бронирования (Reservation): связующая сущность между пользователем, временем и рестораном.

Отзывы (Review): обратная связь от пользователей.

Защита эндпоинтов реализована через **Middleware**. Это позволяет централизованно проверять права доступа до того, как запрос попадет в контроллер.

- **Логика работы:** Middleware перехватывает заголовок Authorization, извлекает **JWT-токен**, проверяет его валидность с помощью

секретного ключа и, в случае успеха, записывает данные пользователя в объект запроса (`req.user`).

- **Типизация:** Для корректной работы с TypeScript был расширен стандартный интерфейс `Request` из библиотеки `Express` (создан кастомный `AuthRequest`), чтобы избежать ошибок при обращении к `req.user`.

Контроллеры реализованы как классы со статическими методами. Это обеспечивает чистоту кода и удобство тестирования.

- **Обработка запросов:** Каждый метод контроллера работает по паттерну `try-catch`. В случае ошибки сервер не «падает», а возвращает клиенту понятный JSON с кодом ошибки (например, 404 Not Found или 400 Bad Request).
- **Интеграция с базой:** Взаимодействие с БД происходит через `AppDataSource.getRepository()`. Например, в `UserController` метод поиска пользователя использует асинхронные функции `async/await` для предотвращения блокировки основного потока (`Event Loop`).

Все пути API вынесены в отдельный модуль `routes`.

- **Структура:** Использован `express.Router()` для группировки эндпоинтов. Это позволяет легко масштабировать проект: пути для пользователей, ресторанов и бронирований разделены по логическим блокам, но объединяются в главном файле `index.ts`.

Вывод

В результате данной работы я написал работающее API